

UNIVERSIDAD DE ANTIOQUIA
DEPARTAMENTO DE INGENIERÍA ELECTRÓNICA Y DE TELECOMUNICACIONES
INFORMÁTICA II



Informe Proyecto Final

Juan David Ballesteros Amaya

Introducción

En el marco del curso de Informática II, se planteó como proyecto final el desarrollo de un videojuego en C++ utilizando el framework Qt. El objetivo fue consolidar los conocimientos adquiridos durante el semestre en temas como Programación Orientada a Objetos (POO), memoria dinámica y herencia.

El videojuego desarrollado, denominado **WaveToWheels**, simula un triatlón de dos etapas: natación y ciclismo. El jugador debe completar ambas etapas en el menor tiempo posible, evitando obstáculos y enemigos en cada nivel. Este proyecto permitió aplicar conceptos físicos reales dentro de un entorno interactivo, demostrando la capacidad de integrar física, gráficos y lógica de juego en una aplicación funcional.

Planteamiento del problema

Se desea desarrollar un videojuego en Qt que esté implementado con una arquitectura limpia utilizando Programación Orientada a Objetos. El juego debe utilizar memoria dinámica en el núcleo del sistema, POO, contar con un mínimo de dos niveles jugables y ofrecer una experiencia visual coherente mediante el uso de sprites.

Objetivos

Objetivo General

Demostrar la capacidad para el desarrollo de aplicaciones no triviales mediante el manejo de C++, Qt y memoria dinámica, así como la disciplina para aprender de forma autónoma los conceptos necesarios para completar el proyecto.

Objetivos Específicos

- Implementar una arquitectura modular basada en Programación Orientada a Objetos.
- Utilizar memoria dinámica para la gestión de objetos en tiempo de ejecución.
- Implementar herencia entre clases definidas por el usuario.
- Desarrollar dos niveles jugables con mecánicas distintas.
- Integrar sprites animados para representar los personajes y el entorno.

Especificación de requerimientos

Requisitos funcionales

- Implementación completa en Qt

- Uso de memoria dinámica para la creación y destrucción de objetos en juego.
- Arquitectura basada en Programación Orientada a Objetos con herencia entre clases propias.
- Dos niveles jugables con mecánicas diferenciadas.
- Diagrama de clases de la capa lógica.
- Sistema de cronómetro que mide el tiempo total de juego.
- Sistema de podio con clasificación según el tiempo registrado.
- Menú principal con instrucciones al jugador.
- Pantallas de transición entre niveles y pantalla final.

Restricciones

- No se permite el uso de librerías externas a Qt o C++ estándar.
- No se delega responsabilidad a código externo o generado automáticamente.
- El manejo de memoria dinámica debe ser explícito y eficiente.

Diseño y arquitectura del sistema

Explicación jerárquica de los componentes

El sistema se estructura en torno a la clase **MainWindow**, que actúa como contenedor principal y coordina la transición entre las distintas pantallas del juego. A partir de esta clase se crean los dos niveles, **Nivel1** y **Nivel2**, que heredan de la clase abstracta **NivelBase**.

La clase **NivelBase** centraliza los elementos comunes a ambos niveles: la referencia a la escena gráfica, el jugador, el fondo animado y los métodos para verificar el estado del nivel. Cada nivel implementa su propia lógica de carga, actualización de enemigos, verificación de colisiones y condición de victoria.

Los enemigos se agrupan bajo la clase abstracta **Enemigo**, de la cual heredan **Tiburón**, **Cangrejo**, **Calamar** y **EnemigoCiclista**. Cada uno implementa un comportamiento particular: patrullaje lateral, persecución con amortiguamiento o movimiento descendente con desplazamiento lateral.

La clase **Jugador** maneja tanto la física del nadador en el nivel 1 como la animación del ciclista en el nivel 2, adaptando su comportamiento según el modo activo mediante el método **establecerSpriteNivel2()**.

Las pantallas de interfaz (**Menu** y **PantallaTransicion**) heredan de **QWidget** e implementan **paintEvent()** para renderizar fondos personalizados.

Desarrollo e implementación

Modelos físicos implementados

El juego incorpora cuatro modelos físicos diferenciados:

1. **Movimiento Parabólico:** El jugador en el nivel 1 combina desplazamiento horizontal constante con una componente vertical sujeta a gravedad reducida (simulando el medio acuático), generando una trayectoria parabólica al impulsar al personaje con la tecla **ESPACIO**.
2. **Movimiento Uniformemente Acelerado (MUA):** El scroll del nivel 2 implementa aceleración constante al presionar **ESPACIO** y desaceleración gradual al soltarlo, siguiendo la ecuación $v = v_0 + at$.
3. **Movimiento con amortiguamiento (Lerp):** El calamar persigue al jugador mediante interpolación lineal: $x_{nuevo} = x + (x_{objetivo} - x) \cdot k$, donde k es un factor de amortiguamiento. Esto produce un movimiento suavizado que se aproxima al objetivo de forma exponencialmente decreciente, similar a un sistema con amortiguamiento crítico. La misma técnica se aplica al seguimiento de cámara en el nivel 1.

Memoria dinámica

La gestión de memoria dinámica se aplica en varios componentes del juego:

- Los objetos del juego (jugador, enemigos, obstáculos, meta) se crean con **new** al iniciar cada nivel.
- `escena->clear()` elimina todos los ítems gráficos al reiniciar o cambiar de nivel.
- Los obstáculos y enemigos del nivel 2 se liberan explícitamente con **delete** en `limpiarElementosFueraDePantalla()` cuando salen de la pantalla.
- Los objetos Qt con padre se eliminan automáticamente cuando su padre es destruido.

Herencia entre clases propias

El proyecto implementa dos jerarquías de herencia entre clases definidas por el usuario:

- **NivelBase** $\rightarrow$ **Nivel1**, **Nivel2**: Herencia de clase abstracta con métodos virtuales puros (`iniciar()`, `pausar()`, `reanudar()`, `terminar()`).
- **Enemigo** $\rightarrow$ **Tiburón**, **Cangrejo**, **Calamar**, **EnemigoCiclista**: Herencia de clase abstracta con `actualizar()` y `cargarFrame()` como métodos virtuales puros.

Mecánicas por nivel

Nivel 1 — Natación: El jugador nada horizontalmente a través de un nivel de 10.000 unidades de longitud. Debe esquivar obstáculos de terreno y enemigos marinos. Los tiburones y cangrejos patrullan lateralmente reduciendo la velocidad del jugador al contacto. El calamar aparece cerca de la meta y persigue al jugador usando interpolación lineal, empujándolo 80 píxeles hacia atrás al colisionar. El fondo animado usa parallax con 16 copias del spritesheet, moviéndose al 30 % de la velocidad del jugador.

Nivel 2 — Ciclismo: El jugador avanza por una carretera con scroll vertical. Puede acelerar con **ESPACIO** (MUA) y moverse lateralmente dentro de los límites de la carretera. Los obstáculos (rocas) y ciclistas rivales descienden continuamente y se generan de forma aleatoria mediante temporizadores. La meta aparece tras un número determinado de repeticiones del fondo.

Sistema de tiempo y podio

El cronómetro se implementa con `QElapsedTimer` en `MainWindow`. Se inicia al comenzar el nivel 1, se pausa durante la pantalla de transición y se reanuda al iniciar el nivel 2. Al finalizar el juego, el tiempo total determina el puesto en el podio:

Puesto	Tiempo
Primer lugar	Menos de 2:22:00
Segundo lugar	Menos de 2:36:00
Tercer lugar	Menos de 2:45:00

Resultados y discusión

El videojuego WaveToWheels funciona correctamente en sus dos niveles, con transiciones fluidas, detección de colisiones, animaciones de sprites y un sistema de cronómetro funcional. La arquitectura modular facilitó el desarrollo incremental, permitiendo depurar cada componente de forma independiente.

Entre las limitaciones identificadas se destacan:

- La detección de colisiones en el nivel 1 utiliza solapamiento de rectángulos, lo que en algunos casos puede producir respuestas imprecisas en esquinas.
- El sistema de generación aleatoria de obstáculos en el nivel 2 puede ocasionalmente generar configuraciones muy difíciles de superar.
- El fondo parallax en el nivel 1 requirió 16 copias estáticas para cubrir el nivel completo, lo que podría optimizarse con un sistema de reciclaje de copias.

Conclusión

El desarrollo de WaveToWheels permitió integrar y consolidar los conocimientos del curso de Informática II: Programación Orientada a Objetos, herencia, memoria dinámica, manejo de eventos y programación gráfica con Qt. La implementación de cuatro modelos físicos dentro de un entorno interactivo demostró que es posible aplicar conceptos de física clásica en el desarrollo de videojuegos de manera efectiva.

El proyecto también representó un reto en términos de arquitectura de software, requiriendo decisiones de diseño que priorizaran la modularidad y la claridad del código sobre la simplicidad inmediata. El resultado es un videojuego funcional que cumple con todos los requisitos planteados y sienta las bases para futuras extensiones.